

Network Intrusion Detection

A four-page report on NSL-KDD five-class classification with Random Forest

Team members	Stefan Stoiana-Mois, Ian Tuttle
Date	23.05.2026
Repository	https://github.com/caramida91/llmtrain.git

0.5005

FINAL MACRO F1

KDDTest+ selected model

0.9169

CV MACRO F1

+/- 0.0374 across folds

0.75

ACCURACY

overall test accuracy

RF-500

BEST MODEL

max_depth=20

Summary

The project focused on supervised intrusion detection using the NSL-KDD dataset. The final Random Forest model improved over the provided baseline macro F1 of about 0.47 and produced a final test macro F1 of 0.5005. The model recognized Normal, DoS, and Probe traffic reasonably well, but struggled with the rare R2L and U2R classes.

1. Approach

The main plan was to start with a clear and reproducible Random Forest baseline, then make controlled changes one at a time. We chose this approach because Random Forest is reliable on tabular data, trains quickly on NSL-KDD, and allows us to focus on the assignment requirement: comparing macro F1 across experiments rather than only reporting accuracy.

Since the competition metric was macro F1, the experiments were judged by how much they helped all five classes equally. This is important because a high accuracy model can still be poor if it ignores rare attacks. For this reason, the report pays special attention to R2L and U2R, even though those classes have much lower support than Normal or DoS.

1.1 Preprocessing and feature handling

Preprocessing steps

The starter-code pipeline was kept mostly unchanged so that the comparison between models remained fair. The categorical columns `protocol_type`, `service`, and `flag` were converted to numeric labels. The original attack names were grouped into the required five categories. The `level` column was dropped, and `num_outbound_cmds` was removed because it was constant zero.

Why no scaling?

No `StandardScaler` or `MinMaxScaler` was used because the final algorithm was tree-based. Decision trees split on thresholds and do not rely on distances in the same way as k-NN, SVM, or neural networks. Keeping the preprocessing simple also reduced the risk of accidentally changing the test distribution.

1.2 Dataset and metric context

Item	Value	Why it matters
Training set	125,973 records	Used for fitting the model and cross-validation.
Test set	22,544 records	Used only for the final evaluation.
Input features	41 original features	Describe connection duration, protocol, content, and traffic behavior.
Target	5 classes	Normal, DoS, Probe, R2L, and U2R.
Main metric	Macro F1	Treats all classes equally, including rare attacks.

Workflow used in the code: load KDDTrain+ and KDDTest+, drop non-feature columns, encode categorical fields, map attack labels into five categories, train Random Forest models, compare macro F1 using cross-validation, then evaluate once on KDDTest+ and save the confusion matrix.

1.2 Class imbalance handling

The hardest part of the assignment was the imbalance between the classes. Normal and DoS records dominate the dataset, while U2R has very few samples. Because macro F1 is an unweighted average over classes, failing on R2L or U2R strongly lowers the final score. We therefore tested whether class weighting could force the model to pay more attention to minority classes.

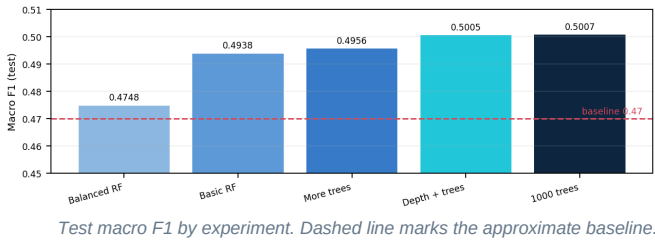
The final training distribution was not modified with oversampling or undersampling. Instead, one experiment used `class_weight='balanced'`, while the other experiments used `class_weight=None`. Surprisingly, the balanced version did not improve the test macro F1-score. It improved the theoretical attention given to rare classes, but it did not translate into better generalization on KDDTest+.

Key lesson

The main issue was not only imbalance, but also generalization. Some attack types in KDDTest+ are not present in KDDTrain+, so simply weighting classes more heavily did not make the model recognize every unseen attack pattern.

2. Experiments

Five main experiments were run. Each experiment changed only one main idea: class weighting, number of trees, or maximum tree depth. This made the results easier to compare and helped identify which changes were useful.



Experiment interpretation

The biggest jump came from moving away from the balanced class-weight experiment and tuning the forest parameters. Increasing the number of trees gave small improvements, while adding a maximum depth of 20 provided the best practical model. The 1000-tree version was technically highest, but only by 0.0002, so the 500-tree version was selected for readability and runtime.

This also shows why a single baseline is not enough. The experiments that looked more theoretically suitable for imbalance were not automatically better on the final unseen test set.

#	Description	Algorithm	Imbalance	CV macro F1	Test macro F1
1	Balanced Random Forest	Random Forest	<code>class_weight='balanced'</code>	0.9064 +/- 0.0392	0.4748
2	Basic Random Forest	Random Forest	None	0.9148 +/- 0.0400	0.4938
3	More trees	Random Forest	None	0.9138 +/- 0.0411	0.4956
4	More trees + limited depth	Random Forest	None	0.9169 +/- 0.0374	0.5005
5	1000 trees + limited depth	Random Forest	None	0.9147 +/- 0.0336	0.5007

Experiment 4 was selected as the final model although Experiment 5 had a marginally higher score. The reason is practical: 1000 trees almost doubled training effort while changing macro F1 from 0.5005 to 0.5007. The difference is too small to be meaningful, so the simpler 500-tree model is the final submitted configuration.

Additional experiment notes

Finding	Explanation
Balanced weighting was not enough	The balanced run gave a test macro F1 of 0.4748. It still did not learn useful recall for R2L and U2R, so weighting alone did not solve the minority-class problem.
More trees helped only slightly	Moving from the basic Random Forest to more trees improved the score from 0.4938 to 0.4956, so the model became a bit more stable but not fundamentally different.
Depth control was the best practical change	Adding <code>max_depth=20</code> improved the selected model to 0.5005. This suggests that limiting tree complexity helped the model generalize better to KDDTest+.

3. Final results

The best practical model was a Random Forest with 500 trees and a maximum depth of 20. It used no class weighting and no additional feature engineering. The final model was reproducible because random_state=42 was used during training and cross-validation.

0.5005

MACRO F1 TEST

final selected score

0.9169

MACRO F1 CV

+/- 0.0374

+0.0305

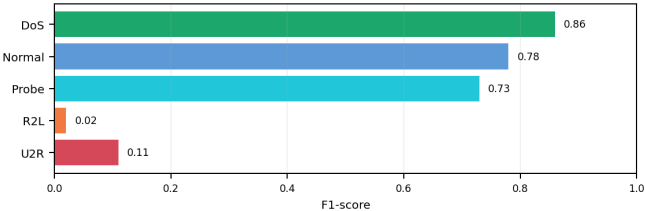
BASELINE GAIN

against about 0.47

Model setting	Final value	Reason
Algorithm	Random Forest	Strong baseline for structured/tabular data.
n_estimators	500	Enough trees for stability without unnecessary runtime.
max_depth	20	Reduced overfitting and improved test score.
class_weight	None	Balanced weighting performed worse on KDDTest+.
random_state	42	Reproducibility.

3.1 Per-class breakdown

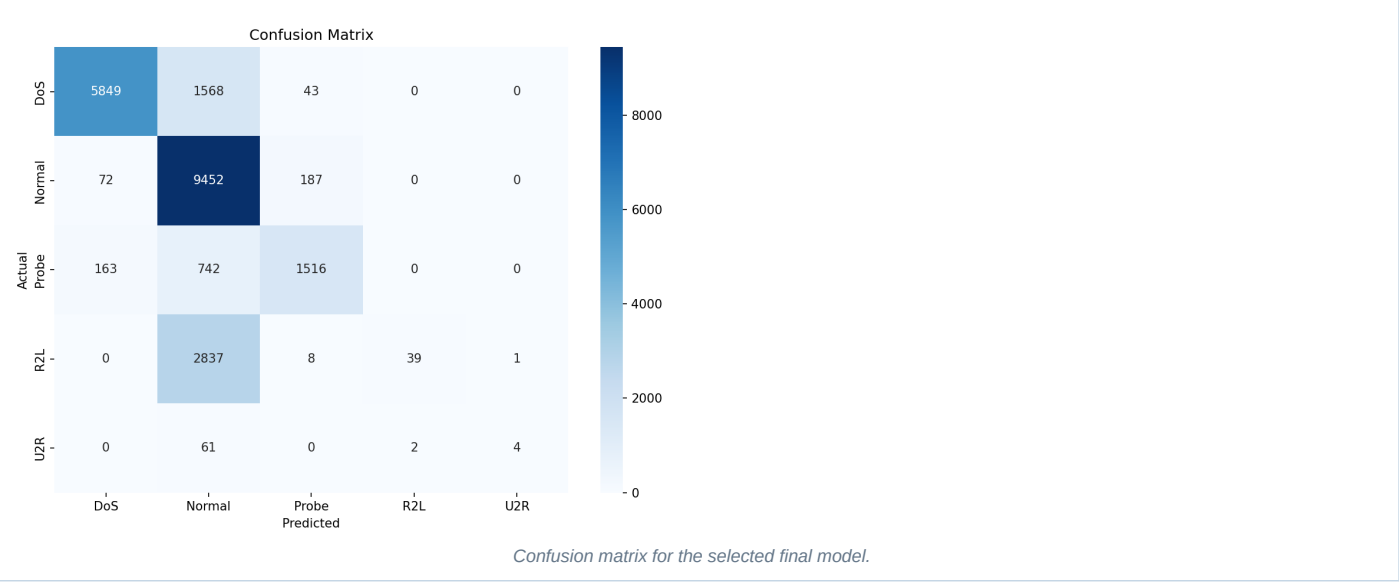
Category	Precision	Recall	F1	Support
DoS	0.96	0.78	0.86	7460
Normal	0.65	0.97	0.78	9711
Probe	0.86	0.63	0.73	2421
R2L	0.97	0.01	0.02	2885
U2R	0.80	0.06	0.11	67



F1-score by class. The low bars show the rare-class weakness.

The classifier was useful for DoS, Normal, and Probe traffic. The main weakness was recall for R2L and U2R: most of those attacks were not detected, so their F1-scores stayed very low. This is the key reason the macro average remained around 0.50 even though the weighted average and accuracy were higher.

3.2 Confusion matrix



Confusion matrix for the selected final model.

The matrix confirms the classification report: many R2L records are predicted as Normal. This means the model often treats remote-to-local attack behavior as legitimate traffic, which is dangerous in real intrusion detection.

4. Cross-validation vs. test score

Cross-validation was performed on the training set using five folds, while the final test score was calculated on KDDTest+. The two numbers are very different, which is important to discuss because it shows how hard it is to generalize from KDDTrain+ to KDDTest+.

Metric	Score
CV macro F1	0.9169 +/- 0.0374
Test macro F1	0.5005
Gap	0.4164

The gap suggests that the model learned patterns from the training distribution very well, but had difficulty when the final test set introduced harder or unseen attack behavior.

Analysis of the gap

A moderate gap is expected because KDDTest+ is intentionally harder and contains attack types that are not present in the training set. However, the gap here is large, so it also suggests some overfitting to the training distribution. Random Forest can memorize complex patterns, especially when many trees are used, and those patterns do not always transfer to new attacks.

The low recall for R2L and U2R is the clearest evidence of this generalization problem. The model learned common traffic patterns, but did not learn enough rare attack behavior to reliably detect minority classes.

5. What worked and what did not

Worked	Did not work	Why it matters
Limiting max_depth to 20	class_weight='balanced'	Depth control helped test performance more than weighting did.
500 trees	1000 trees as final choice	More trees helped, but after 500 the gain was almost zero.
Simple preprocessing	Rare-class recall	The pipeline stayed reproducible, but R2L/U2R need stronger methods.

Reflection and next steps

The final result is above the provided baseline, but the report also shows that improving only common classes is not enough for intrusion detection. In a real security system, missing R2L or U2R attacks would be serious because those categories can represent unauthorized access or privilege escalation. The model should therefore be treated as a starting point, not a production-ready detector.

With more time, we would test XGBoost or LightGBM, use SMOTE or SMOTEENN to increase rare-class representation, tune prediction thresholds to trade some precision for better rare-class recall, and engineer features such as src_bytes/dst_bytes ratios. Another promising direction would be a two-stage system: first detect whether traffic is suspicious, then classify the specific attack category with a model specialized for rare attacks.

Appendix: Environment

Hardware	Intel Core i9-13900H, RTX 5060 8GB Laptop VRAM, 16GB RAM
Python	Python 3.13
Libraries	pandas, numpy, scikit-learn, matplotlib, seaborn, imbalanced-learn, xgboost
Random seed	42

Final conclusion

The final submission is reproducible, includes cross-validation, beats the baseline, and documents both successful and unsuccessful attempts. The strongest result was achieved with a tuned Random Forest rather than class weighting. The main remaining limitation is rare attack detection, especially R2L and U2R, which would require stronger imbalance handling or a more specialized model in future work.

Requirement	Included in report
Approaches tried and why	Yes - Random Forest, class weights, tree count, and depth tuning.
Final macro F1 and classification report	Yes - macro F1 0.5005 and per-class scores.
Confusion matrix	Yes - included as visualization.
CV vs. test comparison	Yes - CV 0.9169 +/- 0.0374 vs. test 0.5005.
Reflection	Yes - future work describes XGBoost/LightGBM, SMOTE, thresholds, and rare-class models.